Um Modelo para Gerenciamento de Perfis de Entidades Através de Inferência em Trilhas

André Wagner

1 Introdução

Atualmente, as aplicações têm se concentrado na coleta de informações a respeito dos usuários, com o objetivo de permitir que elas possam conhecer suas preferências, possibilitando a oferta de serviços personalizados. Assim, estas aplicações organizam seu conhecimento a respeito do usuário em *perfis de usuário*, onde são salvas as informações em um formato baseado em um *modelo de usuário* específico.[1]

Sistemas sensíveis a contexto são aplicações nas quais o contexto atual de um usuário é levado em consideração na disponibilização de serviços. Dey[2] define contexto como qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade, onde uma entidade pode ser uma pessoa, local ou objeto considerado relevante à interação entre o usuário e a aplicação. Desta forma, o *perfil* torna-se parte essencial na descoberta de informações pertinentes aos usuários que possam ser utilizadas na construção de um contexto.

Os ambientes de *computação ubíqua* são parte de um cenário na qual serviços e informações estão disponíveis a qualquer momento, em qualquer lugar[3]. Esta visão vem sendo potencializada pela diminuição do custo e aumento da capacidade de memória e processamento dos dispositivos computacionais, bem como pela cada vez maior disponibilidade de recursos de localização e conectividade. A computação ubíqua enfatiza também a "tecnologia calma", onde a sobrecarga de informação é substituída pela disponibilização de recursos de forma inteligente, tomando a atenção do usuário apenas nos momentos em que é mais relevante.[4]

A capacidade de uma aplicação compreender o contexto de uma entidade pode ser amplificada através do uso de *trilhas*[5]. Uma trilha é o histórico de contextos visitados por uma entidade, e das ações realizadas pela entidade no contexto.

Embora existam alguns trabalhos que utilizam trilhas como forma de armazenar e processar informações de um usuário[6, 5], as trilhas são utilizadas com o objetivo de aquisição de informações para suprir uma necessidade específica do modelo.

Este trabalho busca preencher esta lacuna apresentando um modelo para gerenciamento de perfis, através de inferência em trilhas. O objetivo do modelo é permitir que qualquer aplicação que faça uso de trilhas possa ser capaz de montar seu próprio modelo de perfil e definir as regras necessárias para que este perfil seja inferido a partir das trilhas.

A Seção 2 apresenta conceitos gerais a respeito de gerenciamento de perfis, e na Seção 3 são comparados trabalhos relacionados. A Seção 4 apresenta o modelo, e a Seção 5 conclui comparando o modelo apresentado com os relacionados, e traçando possíveis trabalhos futuros.

2 Conceitos Gerais

A coleta de informações de um usuário pode ser *explícita* ou *implícita*[1]. Na coleta *explícita*, é requisitado ao usuário que preencha informações através de um formulário pré-definido, enquanto na coleta *implícita*, a informação é inferida a partir do estudo do comportamento dos usuários.

Informações a respeito de um usuário colhidas por uma aplicação podem ser úteis para outras aplicações que tenham o mesmo usuário [1]. No entanto, os diferentes *modelos de usuário* utilizados pelas aplicações tornam difícil a utilização de informações de um mesmo *perfil de usuário* de uma aplicação por outra. Além disso, algumas das informações do usuário podem ser válidas apenas dentro de um *contexto* específico, o que dificulta ainda mais que diferentes aplicações compartilharem o mesmo perfil.

Visando resolver estas questões, o conceito de *personalização inter-sistemas* foi introduzido [7] como forma de compartilhar informações de usuários entre diferentes aplicações. Como o maior problema deste tipo de modelo é o alinhamento entre as diferentes abordagens de personalização, o estudo de *sistemas de modelos de usuário* é o principal desafio da pesquisa atual na área de personalização.[1]

O IEEE Standard Computer Dictionary [8] define interoperabilidade como "a capacidade de dois ou mais sistemas ou componentes trocarem informações e utilizar a informação que foi trocada." No âmbito da personalização, interoperabilidade é a capacidade de acessar e interpretar informação derivada de múltiplas fontes heterogênias e integrar esta informação em um modelo de usuário de granularidade própria[9]. Os dois principais modelos de interoperabilidade são: (i) *interoperabilidade sintática*, onde as diferenças entre os modelos de usuário são resolvidos em nível de aplicação, e (ii) *interoperabilidade semântica*, onde as diferenças entre os modelos são resolvidos em nível de conhecimento.

As duas principais abordagens utilizadas em sistemas de modelos de usuários interoperáveis são [1]:

Baseados em padrões : são baseados numa visão *top-down*, definindo um padrão que todas as aplicações envolvidas devem observar;

Baseados em mediação : se baseiam numa visão *bottom-up*, onde ocorre algum tipo de *reconciliação* entre os diferentes modelos de representação.

3 Trabalhos relacionados

Esta seção apresenta trabalhos relacionados a com o modelo proposto, bem como uma comparação entre os mesmos.

3.1 PeLeP

O projeto PeLeP, apresentado por Levis et al[10], busca integrar a utilização de perfis com aperfeiçoamento automático com ambientes de educação ubíqua.

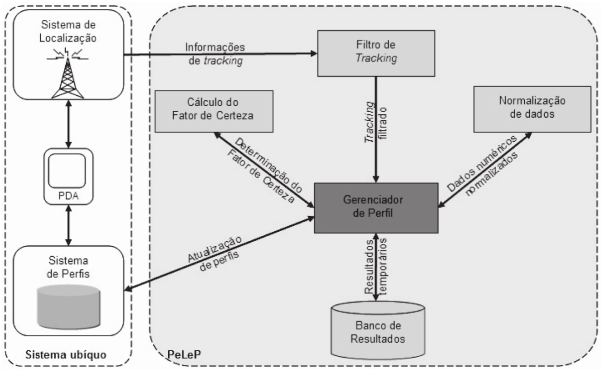

Figura 1: Arquitetura do PeLeP[10]

Barbosa et al compreendem *educação ubíqua* como sendo processos educacionais que ocorrem em qualquer lugar, a qualquer tempo e com qualquer dispositivo, de forma contínua, contextualizada e integrada ao cotidiano do aprendiz. Desta forma, o conhecimento presente no dia-a-dia das mais diferentes formas e em diferentes locais podem ser relacionados com processos educacionais direcionados aos aprendizes [11].

O PeLeP, cujo modelo é ilustrado na Figura 1, é um modelo dedicado ao gerenciamento de perfis de aprendizes em um ambiente ubíquo de ensino e aprendizagem. O modelo administra os perfis automaticamente, inferindo informações através do histórico do aprendiz. As alterações realizadas no perfil refletem o comportamento do aprendiz no ambiente ubíquo, levando em consideração o seu modelo de mobilidade e de contexto.

No PeLeP, o *perfil do aprendiz* consiste em um perfil contendo campos pré-definidos, voltados para o ambiente de educação ubíqua. Os campos são distribuídos nas seguintes categorias: identificação, objetivos, agenda, segurança, relacionamentos, competências e preferências.

É realizado um acompanhamento (*tracking*) do histórico do aprendiz, onde são armazenados os contextos visitados pelo usuário, bem como os conteúdos, dispositivos e aplicativos utilizados em cada contexto. Os valores relativos das variáveis numéricas recebidas no *tracking* são regularizados através de um processo de normalização de amplitude. Como diferentes valores podem ter diferentes impactos para a determinação da preferência de um usuário, calcula-se o Fator de Certeza (FC), de

acordo com a equação 1, onde f_i é o valor normalizado para o fator, p_i o seu percentual, e n o número de fatores considerados.

$$FC = \frac{\sum_{i=0}^{n} f_i p_i}{\frac{100}{n}} \quad (1)$$

O fator tempo também é levado em consideração, onde Fatores de Certeza mais antigos tem um impacto menor. Assim, por exemplo, se o usuário trocar o seu aplicativo preferido, à medida que o tempo for passando o impacto do aplicativo anterior será cada vez menor.

O processo de atualização do modelo do aprendiz é então realizado através de regras pré-definidas, que levam em conta as ocorrências e o seu fator de certeza. As regras suportam conclusões a partir de evidências antecedentes, e cada categoria de informação possui regras diferentes.

O PeLeP foi validado através da integração com o Local (*Location and Context-Aware Learning*), um sistema de educação ubíqua sensível a contexto [12].

3.2 Chellouche et al

O artigo de Chellouche et al[13] discute as dificuldades atuais no desenvolvimento de aplicativos para dispositivos móveis, uma vez que os mesmos possuem uma grande variedade de resolução, memória, poder computacional, entre outros. Além disso, a qualidade da conexão de um usuário pode variar grandemente, e isto também deve ser levado em consideração no desenvolvimento.

O conjunto de informações que caracteriza um usuário é chamada de *perfil de usuário*. Com a intenção de compartilhar o contexto do usuário entre serviços e minimizar as mudanças no desenvolvimento de aplicações sensíveis a contexto, os autores propõem como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O perfil do usuário é constituído por seu contexto, onde a definição de contexto utilizada é aquela dada por Dey[2], na qual contexto refere-se a qualquer informação que pode ser utilizada para caracterizar a situação de uma entidade. Assim, o perfil de usuário é dividido em cinco componentes:

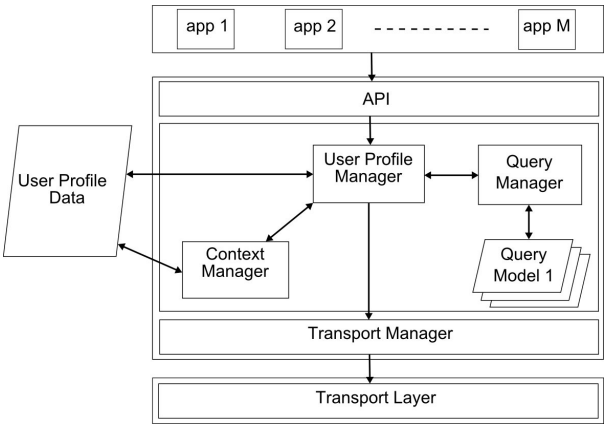

Figura 2: Arquitetura do *middleware* de Chellouche et al[13]

Perfil geral : refere-se às informações básicas, como nome, endereço, telefone, idade, sexo, etc. Inclui também possíveis deficiências pessoais do usuário;

Perfil do dispositivo : informações a respeito dos dispositivos do usuário, tais como marca, capacidade, sistema operacional, entre outros;

Perfil de rede : descreve as redes às quais o usuário pode se conectar, com suas características;

Perfil dos serviços : registra as informações a respeito de serviços, como nome, versão, porta e conteúdo;

Perfil de contexto : contém informações a respeito do ambiente do usuário, como hora, data e localização, aplicações em execução, banda disponível, etc. É considerada a parte dinâmica do perfil, composta por dados voláteis.

O *middleware* é apresentado segundo ilustrado na Figura 2. O *middleware* é composto dos seguintes módulos:

User profile manager : módulo central da arquitetura, que interage com todos os outros módulos com o objetivo de construir o perfil correto;

Context manager : monitora o ambiente, coletando informações com o objetivo de atualizar a parte dinâmica do perfil;

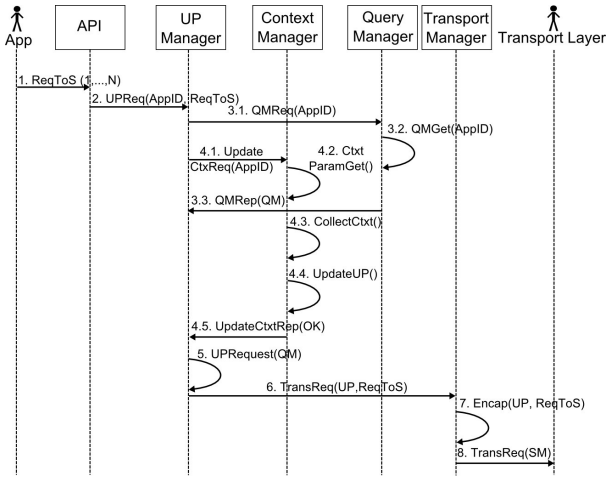

Figura 3: Construção de perfil[13]

Query manager : módulo responsável por validar e salvar um modelo de consulta para cada aplicação;

Transport manager : faz a comunicação do perfil de usuário para a aplicação;

API : constitui o ponto de interface entre as aplicações e o middleware.

O perfil é construído de acordo com a Figura 3. A aplicação solicita ao gerenciador de perfis, através da API, que por sua vez repassa a solicitação ao gerenciador de consultas. Ao mesmo tempo, o gerenciador de contextos é informado que a aplicação está em execução e passa a atualizar o gerenciador de perfis com as informações atualizadas. O perfil de usuário é criado e enviado ao gerenciador de transporte, que retorna o perfil solicitado à aplicação solicitante.

3.3 UUCM

O artigo apresentado por Niederée et al[7] busca responder três indagações, que são apresentadas como sendo os três desafios básicos da personalização:

- Como modelar o usuário e sua situação de modo extensível e unificado, de forma que este modelo possa ser interpretado e usado para personalização por diferentes sistemas?
- Como coletar dados para preencher e atualizar perfis de usuário a partir das interações do usuário com diferentes sistemas baseados em um modelo de contexto de usuário unificado?
- Como habilitar conceitualmente e tecnicamente diferentes sistemas a fazer uso dos perfis de usuário coletados para aperfeiçoamento do suporte ao usuário?

Os autores apresentam como resposta a estas indagações um modelo denominado UUCM, que tem como objetivo modelar características do usuário e de seu contexto. O conceito de contexto utilizado é aquele apresentado por Berenetti et al[14], na qual o contexto pode ser pensado como as informações "extra", geralmente implícitas (como associações, fatos, suposições), que podem ser utilizadas para compreender plenamente uma interação, comunicação ou representação de conhecimento.

A abordagem para a modelagem unificada de contextos de usuário é proposta em dois níveis. No nível *abstrato*, são definidos os blocos básicos de construção: contexto de usuário, facetas do modelo de usuário, propriedades principais e dimensões do modelo de usuário. O termo *faceta* representa as diferentes características do usuário. Este nível define um metamodelo para as dimensões e facetas concretas utilizadas na descrição do modelo de contexto de usuário. Para que a abordagem de personalização multi-sistema seja válida, é necessário que este metamodelo de contexto de usuário seja publicado como uma ontologia compartilhada, e que todos os participantes utilizem este modelo.

No nível *concreto*, um conjunto de dimensões e facetas são definidos. As facetas e dimensões são descritas como parte de uma ontologia adicional que é compartilhada pelos componentes envolvidos na personalização multi-sistemas.

O modelo do UUCM é apresentado na Figura 4. Segundo o modelo, um perfil de usuário (*Unified User Context Model*) é dividido em um conjunto de contextos de trabalho (*Working Contexts*), onde cada contexto é descrito por um conjunto de facetas (*UUCM facet*). Cada faceta é vinculada a uma dimensão (*UUCMDimension*).

O UUCM apenas define o método através do qual o perfil de contexto de usuário é descrito e estruturado para personalização multi-sistema. Para a descrição de perfis de contexto concretos, o UUCM

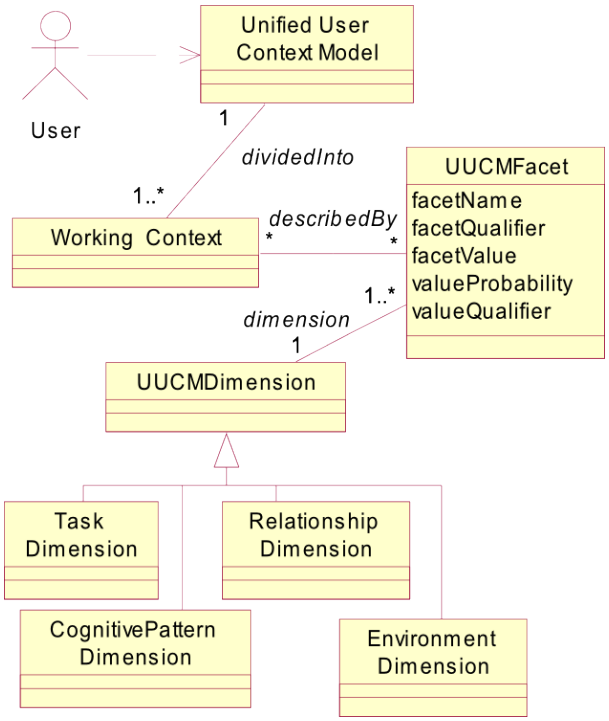

Figura 4: Modelo do UUCM[7]

utiliza ontologias (ou vocabulários) para as dimensões, facetas e valores de faceta.

As seguintes dimensões são utilizadas:

Cognitiva (*CognitivePattern*): descreve as dimensões cognitivas do usuário, tais como *áreas de interesse*, *competências* e *preferências*;

Tarefa (*Task*): descreve os objetivos do usuário quando ele interage com sistemas. Utiliza dados tais como *tarefa atual*, *papel do usuário na tarefa* e *histórico da tarefa*;

Relacionamentos (*Relationship*): armazena as entidades com as quais o usuário tem alguma relação;

Ambiente (*Environment*): parâmetros que são tipicamente utilizados para abordagens sensíveis a contexto, tais como *tempo*, *localização*, *dispositivo* e *linguagem*.

O perfil de usuário do UUCM é codificado como um *RDF Schema*[15], ampliado com expressões *OWL*.

O modelo inclui ainda o conceito de *passaporte de contexto*. O passaporte de contexto é uma representação compacta do modelo de contexto atual do usuário para a personalização multi-sistemas. Ele contém informações ontologicamente arranjadas a respeito dos padrões cognitivos (habilidades, interesses, etc), o ambiente (data, local, dispositivo usado) e sua rede pessoal de pessoas e relacionamentos envolvidos.

Para utilizar o passaporte de contexto em um ambiente de personalização multi-sistemas, o usuário "apresenta" o passaporte ao sistema que o usuário deseja utilizar. Como o passaporte está vinculado a uma ontologia compartilhada, é possível que o sistema consiga interpretar parcialmente o passaporte através de uma arquitetura mediadora. Como resultado, dois fluxos são possíveis: (i) do passaporte para o sistema, auxiliando o sistema a compreender o contexto do usuário, e (ii) do sistema para o passaporte, realizando alterações nas informações de contexto do usuário.

3.4 Avaliação dos Ambientes Pesquisados

O PeLeP[10] é um modelo que suporta o aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua. Ele usa o histórico dos aprendizes nos contextos de um ambiente ubíquo para atualização dos perfis. O perfil do aprendiz no modelo é refinado e enriquecido através de inferências. As inferências são baseadas na mobilidade do aprendiz e no monitoramento das tarefas que ele executa nos contextos de um ambiente de educação ubíqua.

O modelo apresentado por Chellouche et al[13] apresenta um *middleware* voltado para aplicações móveis que pode ser usado por diferentes aplicativos como um servidor central de perfis. Neste modelo, é proposto como solução um *middleware* que gerencia o perfil de um usuário dinamicamente e supre às aplicações todas as informações que elas podem precisar durante o período de adaptação.

O UUCM[7] é um modelo que pode ser utilizado para modelar as características de um usuário e seu contexto em ambientes de personalização de múltiplos sistemas. A proposta diz respeito a uma ontologia básica, baseada em contexto, a qual os sistemas podem estender, desde que os sistemas que

usem este modelo concordem em uma ontologia básica.

Embora os modelos apresentados atualizem parte do perfil como forma de oferecer suporte a sensibilidade a contexto, nenhum deles permite que os aplicativos criem novos campos nem definam as regras para atualização dos campos existentes.

A Tabela 1 exibe um comparativo dos trabalhos apresentados, onde os seguintes fatores são levados em consideração:

Interoperabilidade: o modelo apresenta interoperabilidade – caso positivo, se a mesma é sintática ou semântica;

Perfis dinâmicos: o modelo de perfil permite que os perfis sejam atualizados de acordo com eventos externos, sem a interferência do usuário;

Armazenamento: define onde ficam armazenados os perfis do usuário;

Sensível a contexto: o contexto do usuário é levado em consideração na formação do perfil;

Utiliza trilhas: as trilhas do usuário são levadas em consideração na construção do perfil;

Domínio: o perfil é voltado para um domínio específico, ou se o mesmo permite que vários domínios possam ser utilizados.

4 eProfile

O eProfile é um gerenciador de perfis voltado para ambientes distribuídos, que infere dados de perfil através da inferência das trilhas de uma entidade. A trilha é disponibilizada por um gerenciador de trilhas que se conecta ao eProfile, onde é feita uma mediação entre o formato de dados da trilha e do eProfile.

A figura tradicional do usuário na personalização é extendida aqui pelo conceito de *entidade*. Uma entidade é um conceito mais amplo que o de um usuário, não representando apenas pessoas mas também objetos que possam criar uma trilha e ter um perfil, como um veículo ou um dispositivo móvel. Uma entidade pode também ser composta por um grupo de entidades como, por exemplo, uma frota de caminhões onde cada caminhão tem seu perfil único, e onde a frota tem um perfil também.

A ontologia de perfil do eProfile pode ser extendida pelas aplicações, de modo que as aplicações possam armazenar dados de perfil que interessem apenas a si mesmos. Assim, as aplicações podem disponibilizar suas próprias regras para a inferência das informações relacionadas à sua própria estrutura de dados. Os dados inferidos ficam então disponíveis para serem consultados pelas entidades.

Como o eProfile é um modelo onde é necessário inferir informações de forma autônoma a respeito de dados armazenados em uma ontologia, a tecnologia de agentes autônomos foi escolhida para o desenvolvimento.

4.1 Modelo

A Figura 5 apresenta o modelo do eProfile.

As *aplicações* são os aplicativos que são utilizados pelas entidades para se conectar aos gerenciadores de trilhas e perfil. Neste modelo, a aplicação se conecta a um gerenciador de trilhas genérico, que é responsável pelo armazenamento da trilha da entidade. O perfil da entidade será inferido a partir das trilhas registradas no gerenciador.

As trilhas das entidades são buscadas do gerenciador de trilhas pelo eProfile. Como o eProfile pode se conectar a diferentes gerenciadores de trilha (Figura 6), onde em cada um deles a trilha tem um formato específico, é necessário que seja realizada uma conversão entre o formato original e o formato utilizado pelo eProfile, que será apresentado mais adiante. Este trabalho é realizado por um agente entitulado *conversor*, que é capaz de realizar a conversão para os formados de trilha suportados pelo eProfile.

Desta forma, o eProfile não tem a necessidade de conhecer a aplicação – mas precisa conhecer o formato utilizado pelo gerenciador de trilhas.

A trilha do usuário é então armazenada dentro do eProfile em uma *ontologia de trilha*. Esta ontologia possui as classes básicas para a construção de uma trilha, sendo que a mesma pode ser extendida de acordo com a especificação do gerenciador de trilhas que está sendo utilizado. A forma como ocorre esta extensão é detalhada na Seção 4.2.

Cada aplicação tem a possibilidade de montar seu próprio perfil. Para que o eProfile processe este perfil, ele precisa de três informações: (i) a trilha

Trabalho	Interope-rabilidade	Perfis dinâmicos	Armaze-namento	Sensibilidade a contexto	Utiliza trilhas	Domínio
PeLeP [10]	Não	Sim	Servidor	Sim	Parcial	Educação
Chellouche et al [13]	Sintática	Sim	Servidor	Sim	Não	Serviços multimídia
UUCM [7]	Sintática	Sim	Cliente	Sim	Não	Genérico

Tabela 1: Comparação entre os trabalhos apresentados

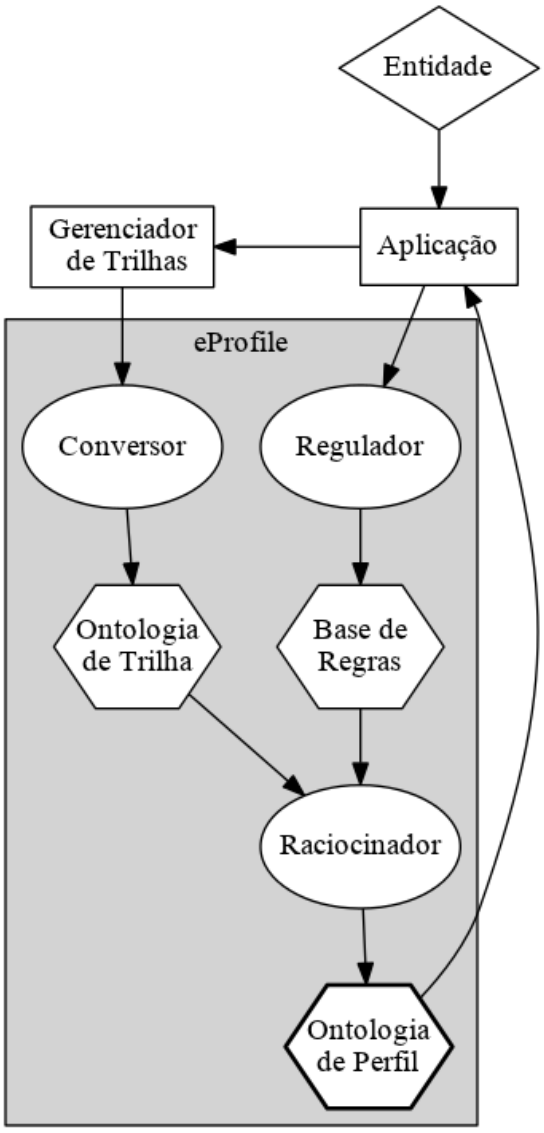

Figura 5: Modelo do eProfile, com as setas representando o fluxo de dados

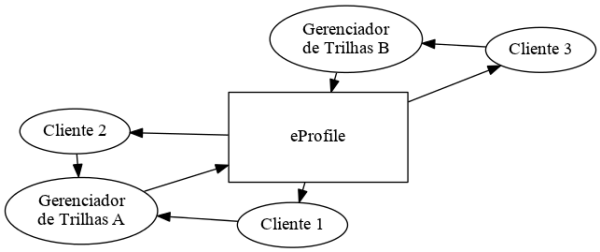

Figura 6: eProfile conectado a mais de um gerenciador de trilhas

da entidade, (ii) o modelo de perfil da aplicação e (iii) as regras de inferência para o perfil. A trilha da entidade é carregada pelo eProfile a partir do gerenciador de trilhas, enquanto as outras duas informações são enviadas diretamente da aplicação para o eProfile. Desta forma, as regras de inferência definidas pela aplicação serão utilizadas para inferir seu próprio perfil.

O agente responsável pela recepção das regras é o *regulador*. O regulador tem a função de receber as regras, validá-las e inserí-las na *base de regras*. Juntamente com as regras, é especificado a criticidade da informação. O sistema utilizará esta informação para determinar com que freqüência a inferência desta informação será executada.

Uma vez de posse das trilhas e das regras, o agente *raciocinador* realiza a inferência da trilha, gerando assim a *ontologia de perfil*. Esta ontologia, da mesma forma que a ontologia de trilha, contém classes básicas a partir das quais as aplicações vão especializar, utilizando-as para armazenar informações que sejam úteis a eles. A aplicação pode então conectar-se ao eProfile e buscar o perfil de sua entidade. A Figura 5 exemplifica isso, onde a seta que conecta a Ontologia de Perfil (no eProfile) à aplicação mostra o sentido do fluxo de dados.

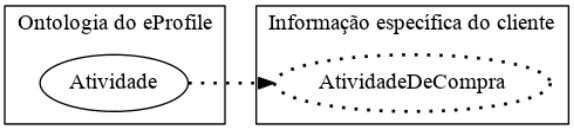

Figura 7: Exemplo da extensão de uma classe do eProfile

4.2 Ontologias

O eProfile utiliza internamente duas ontologias: uma ontologia de trilhas e uma ontologia de perfis. Para o armazenamento e processamento das ontologias, foram estudadas algumas linguagens de representação de conhecimento CycL[16], RDF/S[17], DAML+OIL[18] e OWL[18].

Devido à expressividade e ao suporte de ferramentas externas, o OWL foi escolhido como a linguagem de ontologias utilizado pelo eProfile para modelagem da ontologia do perfil.

As ontologias utilizadas pelo eProfile – tanto a ontologia de trilhas quanto a ontologia de perfil – são ontologias básicas, extendidas através de subclasses de acordo com o formato da trilha enviada pelo gerenciador (no caso da ontologia de trilha) ou das necessidades da aplicação (no caso da ontologia do perfil). Isto é representado na Figura 7, onde o eProfile oferece uma classe genérica "Evento", que é especializada pela classe "EventoDeCompra", utilizada por um dos gerenciadores de trilha.

4.2.1 Ontologia de Trilha

O eProfile implementa uma ontologia de trilha. Esta ontologia será alimentada a partir dos diferentes gerenciadores de trilhas suportados, onde o agente *conversor* é responsável por receber os dados no formato do gerenciador de trilhas e convertê-los para a ontologia de trilhas do eProfile. Assim, o raciocinador terá uma ontologia única sobre a qual poderá raciocinar.

A ontologia de trilha está representada na Figura 8, onde todos os relacionamentos, representados pelas setas, são do tipo "has-some" (tem-algum(s)).

Abowd e Dey definem que aplicações sensíveis a contexto estão interessadas em saber *quem*, *onde*, *quando* e *o quê* (isto é, o que o usuário está fazendo), de forma a poderem descobrir por que a situação está ocorrendo[19]. Assim, uma trilha é representada por um conjunto de eventos caracterizados pelas seguintes classes:

Indentity: É a identidade da entidade que está realizando a ação. Esta entidade pode estar realizando a ação com um ou mais recursos (*Resource*) (como um aplicativo ou dispositivo) e pode estar vinculada a um perfil (*Profile*) dentro do eProfile. A classe **Participants** contém as identidades das entidades que estão participando deste evento;

Location: Onde está ocorrendo a ação. Uma localização pode ser uma localização física específica (como uma sala) ou não (uma estrada, por exemplo). A localização pode estar vinculada a uma região (*Region* (que, adicionalmente, pode estar vinculada a outras subregiões) e geralmente possui uma localização global (GPS) vinculada a ela. Outros tipos de localização podem ser extendidas a partir da classe;

Time: Reponde à questão de *quando* ocorreu o evento;

Activity: Refere-se à ação que o usuário está executou (ou tentou executar). É a classe mais abrangente da ontologia, e a que possivelmente será a mais especializada pelos gerenciadores de trilha.

4.2.2 Ontologia de Perfil

A ontologia de perfil é a ontologia utilizada para armazenar as informações de perfil inferidas a partir da trilha. Esta ontologia está representada na Figura 9.

A ontologia possui duas classes básicas: *Entity* (entidade) e *Information* (informação). Assim, os perfis de várias entidades podem ser armazenados nesta ontologia, onde cada perfil possui um conjunto de informações. Este conjunto de informações será extendido pelas aplicações através de herança (subclasses).

Cada entidade é identificada por um nome único, mesmo que o perfil seja alimentado por várias aplicações. Desta forma, é possível que uma aplicação tenha acesso a informações de perfil gravadas por outra aplicação, desde que ela tenha conhecimento das classes utilizadas pelo outro.

As informações podem ser de três tipos:

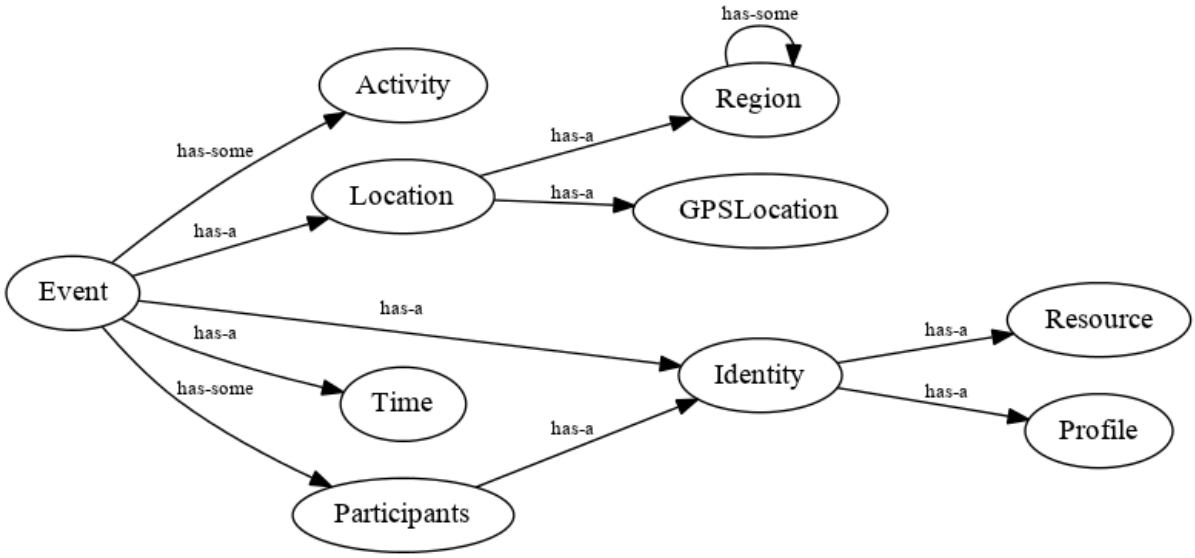

Figura 8: Ontologia de Trilha do eProfile

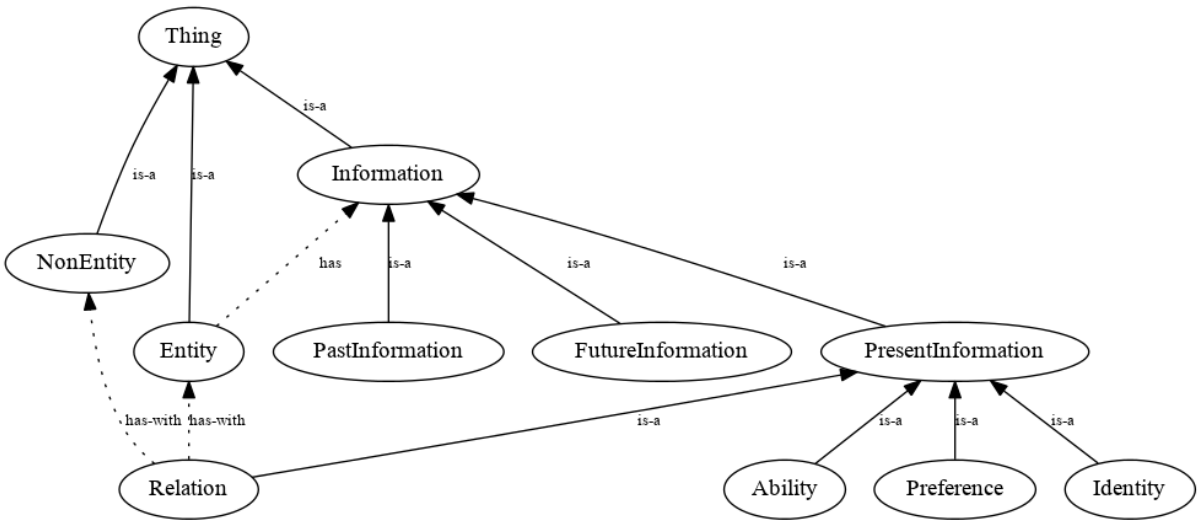

Figura 9: Ontologia de Perfil do eProfile

PastInformation: refere-se a eventos que ocorreram no passado. As informações armazenadas nesta classe são, de um modo geral, cumulativas. Exemplos seriam as escolas onde uma pessoa estudou, ou os destinos onde um caminhão já entregou alguma carga;

PresentInformation: são informações atuais da entidade, subdivididas em subclasses descritas mais adiante;

FutureInformation: são informações previstas a respeito do futuro de uma entidade, como o conjunto de disciplinas que uma pessoa tem a cursar, ou quantos quilômetros um caminhão pode percorrer antes de reabastecer.

As informações atuais podem ser dos seguintes tipos:

Identity: informações que identificam a entidade, como dados cadastrais.

Preference: preferências da entidade. Podem ser informações a respeito de interesses de um usuário ou aspectos mais técnicos, como protocolo preferencial de comunicação, ou até mesmo informações compostas, como qual qualidade de vídeo utilizar para cada banda de comunicação;

Ability: conjunto de capacidades de uma entidade. Capacidades incluem tanto habilidades de um usuário (como jogar xadrez) quanto capacidades de um serviço (por exemplo, suporte a webservices SOAP);

Relation: conjunto de relações relevantes desta entidade. As relações podem dizer respeito a uma outra entidade (por exemplo, um usuário que tenha outros amigos ou um carro que tem um dono) ou uma não-entidade (como um usuário que tem um dispositivo). No caso de relações com outras entidades, se a entidade possuir um perfil cadastrado, é possível realizar uma vinculação.

Todos as extensões (subclasses) da ontologia são definidas pelas aplicações, e todas as informações do perfil são inferidas. A forma como esta inferência ocorre é descrita na Seção 4.4.

4.3 Agentes

4.4 Regras e Inferência

Para que uma aplicação possa utilizar os serviços oferecidos pelo eProfile, é necessário que a aplicação envie ao eProfile a ontologia de perfil que ele irá utilizar, extendendo a ontologia do eProfile. A ontologia é enviada quando a aplicação faz seu primeiro acesso ao eProfile, e pode ser atualizada posteriormente. Ela deve ser enviada em formato OWL utilizando a sintaxe RDF/XML[15].

A Figura 10 mostra um exemplo de uma ontologia de perfil. Neste caso, em um perfil pessoal a classe `Identity` está sendo extendida para a classe `PersonalInformation`, que tem as propriedades `hasName` e `hasAge`. Este exemplo mostra o OWL/XML gerado pelo editor de ontologias Protégé[20]. As linhas 19 e 14 importam a ontologia do eProfile e criam um *namespace* para ele; nas linhas 21 a 23 é criada a nova classe que extende da classe `Identity` do eProfile.

4.4.1 Regras de Inferência

A aplicação pode, a qualquer momento, criar, editar ou excluir regras para a inferência das informações de sua trilha, de modo a gerar informações úteis dentro da ontologia de perfil.

Existem dois tipos de regras que podem ser criadas:

Regra em OWL: uma regra é criada em OWL, extendendo a ontologia de trilha;

SPARQL: uma regra é criada em SPARQL (linguagem de consulta de ontologias[21]), onde a consulta é executada contra a ontologia de trilha;

SWRL: uma regra é criada em SWRL (linguagem de regras[22]), que será executada sobre a ontologia de trilha.

A nova regra é gravada em uma base de regras. Assim, regras podem ser compartilhadas entre diferentes aplicações, desde que ambas extendam da mesma forma a ontologia de trilha.

Quando uma nova regra é enviada ao eProfile, ela deve conter as seguintes informações:

Nome: o nome da regra, para que possa ser atualizada ou excluída posteriormente;

```
<?xml version="1.0"?>
<!DOCTYPE rdf:RDF [
    <!ENTITY owl "http://www.w3.org/2002/07/owl#" >
    <!ENTITY person "http://www.example.com/person#" >
    <!ENTITY xsd "http://www.w3.org/2001/XMLSchema#" >
    <!ENTITY rdfs "http://www.w3.org/2000/01/rdf-schema#" >
    <!ENTITY rdf "http://www.w3.org/1999/02/22-rdf-syntax-ns#" >
    <!ENTITY trail "http://code.google.com/p/eprofile/trail.owl#" >
]>
<rdf:RDF xmlns="http://www.example.com/person#"
     xml:base="http://www.example.com/person"
     xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#"
     xmlns:person="http://www.example.com/person#"
     xmlns:trail="http://code.google.com/p/eprofile/trail.owl#"
     xmlns:owl="http://www.w3.org/2002/07/owl#"
     xmlns:xsd="http://www.w3.org/2001/XMLSchema#"
     xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <owl:Ontology rdf:about="http://www.example.com/person">
        <owl:imports rdf:resource="http://code.google.com/p/eprofile/trail.owl"/>
    </owl:Ontology>
    <owl:Class rdf:about="&person;PersonalInformation">
        <rdfs:subClassOf rdf:resource="&trail;Identity"/>
    </owl:Class>
    <owl:DatatypeProperty rdf:about="&person;hasAge">
        <rdfs:range rdf:resource="&xsd;positiveInteger"/>
    </owl:DatatypeProperty>
    <owl:DatatypeProperty rdf:about="&person;hasName">
        <rdfs:range rdf:resource="&xsd;string"/>
    </owl:DatatypeProperty>
</rdf:RDF>
```

Figura 10: Extensão de uma classe no eProfile

Tipo de regra: se é uma regra em OWL, SPARQL ou SWRL;

Regra: a regra propriamente dita, na linguagem escolhida;

Objeto: classe, instância ou propriedade da ontologia de perfil que será atualizada;

Instrução de atualização: a forma como a instrução será atualizada;

Freqüência de atualização: a freqüência com que a regra deve ser atualizada;

Validade: a data limite até quando a regra será processada (pode ser ilimitada).

A instrução de atualização pode ser de um dos seguintes tipos:

Substituição: o item anterior é substituído por um novo item. Pode ser utilizado, por exemplo, em dados cadastrais como endereço e telefone, onde cada novo item substitui o anterior;

Adição: uma informação é adicionada, como a lista de endereços visitados por um carro;

Expressão: uma informação que precisa ser calculada. Neste caso, a própria aplicação passa a expressão do cálculo. Calcular o percentual de interesse de um usuário por um determinado assunto, por exemplo, seria uma informação deste tipo.

Uma vez que a regra tenha sido validada pelo agente regulador, ela entra em funcionamento e passa a ser elegível para execução.

Tanto as consultas via OWL quanto as consultas via SPARQL são executadas através de um motor de inferências externo. Foram estudados os seguintes raciocinadores que oferecem suporte a estas operações: Pellet[23], FaCT++[24] e HermiT[25].

Devido ao conjunto de funcionalidades e ao desempenho superior, o HermiT foi escolhido como o raciocinador utilizado pelo eProfile.

5 Conclusão

Referências

[1] M. Viviani, N. Bennani, and E. Egyed-Zsigmond, "A Survey on User Modeling in Multi-Application Environments," in *The Third International Conference on Advances in Human-oriented and Personalized Mechanisms, Technologies, and Services CENTRIC'10*, pp. 111–116, IEEE, Aug. 2010.

[2] A. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 6, no. 1, pp. 4–7, 2001.

[3] M. Weiser, "The computer for the 21st century," *Scientific America*, pp. 94–104, 1991.

[4] M. Weiser and J. S. Brown, "Designing calm technology," *PowerGrid Journal*, July 1996.

[5] J. Silva, J. Rosa, J. Barbosa, D. N. F. Barbosa, and L. A. M. Palazzo, "Content distribution in trial-aware environments," in *Proceedings of the XV Brazilian Symposium on Multimedia and the Web*, WebMedia '09, (New York, NY, USA), pp. 15:1–15:8, ACM, 2009.

[6] D. M. da Silva, J. Barbosa, and R. Vieira, "Octopus: um modelo para classificação de perfil de usuários usando trilhas em sistemas ubíquos," in *Companion Proceedings of the XIV Brazilian Symposium on Multimedia and the Web*, WebMedia '08, (New York, NY, USA), pp. 185–188, ACM, 2008.

[7] C. Niederée, A. Stewart, B. Mehta, and M. Hemmje, "A Multi-Dimensional, Unified User Model for Cross-System Personalization," in *E4PIA Workshop 2004*, 2004.

[8] IEEE, "IEEE standard computer dictionary. a compilation of IEEE standard computer glossaries," *IEEE Std 610*, p. 1, 1991.

[9] F. Carmagnola and F. Cena, "User identification for cross-system personalisation," *Inf. Sci.*, vol. 179, pp. 16–32, January 2009.

[10] D. Levis, J. L. V. Barbosa, and S. C. C. S. Pinto, "Aperfeiçoamento automático do perfil do aprendiz em ambientes de educação ubíqua," *Revista Brasileira de Informática na Educação*, vol. 16, no. 1, 2008.

[11] D. Barbosa, C. Geyer, and J. Barbosa, "Globaledu - an architecture to support learning in a pervasive computing environment," in *New Trends and Technologies in Computer-Aided Learning for Computer-Aided Design* (A. Rettberg and C. Bobda, eds.), vol. 192 of *IFIP International Federation for Information Processing*, pp. 1–10, Springer Boston, 2005.

[12] J. Barbosa, R. Hahn, S. Rabello, and D. Barbosa, "Local: a model geared towards ubiquitous learning," *SIGCSE Bull.*, vol. 40, pp. 432–436, March 2008.

[13] S. A. Chellouche, J. Arnaud, and D. Négru, "Flexible user profile management for context-aware ubiquitous environments," in *Proceedings of the 7th IEEE conference on Consumer communications and networking conference*, CCNC'10, (Piscataway, NJ, USA), pp. 980–984, IEEE Press, 2010.

[14] M. Benerecetti, P. Bouquet, and C. Ghidini, "Contextual reasoning distilled," *Philosophical Foundations of Artificial Intelligence. A special issue of the journal of Experimental and Theoretical AI (JETAI)*, vol. 12, no. 3, pp. 279–305, 2000.

[15] W3C, "RDF/XML syntax specification (revised)," 2011. http://www.w3.org/RDF/.

[16] D. B. Lenat and R. V. Guha, *Building Large Knowledge-Based Systems; Representation and Inference in the Cyc Project.* Boston, MA, USA: Addison-Wesley Longman Publishing Co., Inc., 1st ed., 1989.

[17] W3C, "RDF vocabulary description language 1.0: Rdf schema," 2011. http://www.w3.org/TR/rdf-schema/.

[18] M. C. Daconta, L. J. Obrst, and K. T. Smith, *The Semantic Web : A Guide to the Future of XML, Web Services, and Knowledge Management.* Indianapolis: Wiley, May 2003.

[19] G. D. Abowd, A. K. Dey, P. J. Brown, N. Davies, M. Smith, and P. Steggles, "Towards a better understanding of context and context-awareness," in *Proceedings of the 1st international symposium on Handheld and Ubiquitous Computing*, HUC '99, (London, UK), pp. 304–307, Springer-Verlag, 1999.

[20] S. C. for Biomedical Informatics Research (BMIR), "The protégé ontology editor and knowledge aquisition system," 2011. http://protege.stanford.edu/.

[21] W3C, "SPARQL query language for RDF," 2008. http://www.w3.org/TR/rdf-sparql-query/.

[22] W3C, "SWRL: A semantic web rule language combining OWL and ruleml," 2004. http://www.w3.org/Submission/SWRL/.

[23] E. Sirin, B. Parsia, B. Grau, A. Kalyanpur, and Y. Katz, "Pellet: A practical OWL-DL reasoner," *Web Semantics: Science, Services and Agents on the World Wide Web*, vol. 5, pp. 51–53, June 2007.

[24] D. Tsarkov and I. Horrocks, "Fact++ description logic reasoner: System description," in *Proc. of the Int. Joint Conf. on Automated Reasoning (IJCAR 2006)*, vol. 4130 of *Lecture Notes in Artificial Intelligence*, pp. 292–297, Springer, 2006.

[25] B. Motik, R. Shearer, and I. Horrocks, "Hypertableau Reasoning for Description Logics," *Journal of Artificial Intelligence Research*, vol. 36, pp. 165–228, 2009.